Agreement as a Reusable Primitive for LLM Agents: Skill vs. Harness

Weekly Research Report

Your Name

Institute of Science Tokyo

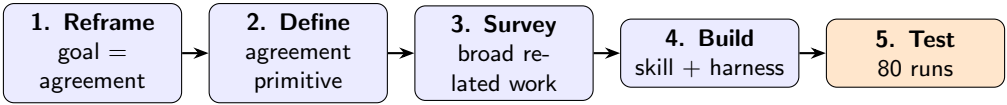
October 29, 2026

Description of the Meeting Last Week (Oct 23)

What I presented: where to enforce coordination (prompt vs. harness) for parallel coding agents

Feedback	What it asks for
The goal is **agreement**; software is only one use case	domain-independent problem
Use cases first; several, not only software	motivating examples
Build a tool with an abstract interface	a reusable primitive
Part skill, part harness; compare with / without	a skill $\times$ harness comparison
When is agreement needed, and how does it help?	a characterization
Agreement $\rightarrow$ output $\rightarrow$ evaluation	a pipeline view
Too software-centric; metrics can come later	reframe the story

Progress in This Week: Overview



Item	Output	Status
Problem and use cases	5 use cases, characterization axes	draft
Definition	interface + 5 properties	draft
Related work	5 areas, positioning	done
Prototype	`SKILL.md`, `agree.py`, 3 harness hooks	working
Experiment	coding task, 8 conditions $\times$ 10 runs	pilot done

Background and Motivation

Background: agents with different roles must make the same **shared choice** τ; each option is valid, but mixing options breaks the result.

Use case	Shared choice τ
Software	interface (types, error signal)
Data analysis	outlier rule
Scheduling	meeting slot
Writing	terms, figures
Operations	image version

Motivation

- Two coding agents: $\sim$25% success, about half of one agent; top failures are work overlap (33%) and divergent architecture (30%) [1]
- Agents break commitments even when they talk [1]
- Constraints dropped after context compaction: violations 0/100 $\rightarrow$ 30/100 [2]

$\Rightarrow$ agreeing once is not enough; outputs must **stay** consistent

[1] A. Khatua, H. Zhu, P. Tran, et al. CooperBench: Why Coding Agents Cannot be Your Teammates Yet. *COLM*, 2026. [2] Z. Wang, Y. Zhang, D. Lee, N. Baracaldo, Y. Yang. Lost in Context Compaction: When LLM Systems Drop Side Constraints. arXiv:2608.11242, 2026 (preprint).

Related Work: Five Areas

Area	What it gives us	What is missing for us
Distributed consensus & commit [3,4,5]	agreement, validity, termination; commit/abort	assumes deterministic replicas
Norms & commitments in MAS [6,7]	monitoring compliance; regimentation vs. enforcement	norms written by designers, not LLMs
LLM agreement / consensus [8,9]	LLMs agreeing on a value or answer	agreement ends at the answer
LLM contracts & coordination [10,11,12]	contracts, merge, runtime control	one enforcement point each
Skills & harness [13,14]	skills help; harness enforces rules	no skill vs. harness comparison

[3] L. Lamport, R. Shostak, M. Pease. The Byzantine Generals Problem. *ACM TOPLAS*, 1982. [4] P. A. Bernstein, V. Hadzilacos, N. Goodman. *Concurrency Control and Recovery in Database Systems*. 1987. [5] F. B. Schneider. Implementing Fault-Tolerant Services Using the State Machine Approach. *ACM CSUR*, 1990. [6] M. P. Singh. An Ontology for Commitments in Multiagent Systems. *AI and Law*, 1999. [7] D. Grossi, H. Aldewereld, F. Dignum. Ubi Lex, Ibi Poena: Designing Norm Enforcement in E-Institutions. *COIN*, LNCS, 2007. [8] F. Berdoz, L. Rugli, R. Wattenhofer. Can AI Agents Agree? arXiv:2603.01213, 2026. [9] C. Ruan, Y. Wang, Z. Shi, J. Li. Reaching Agreement Among Reasoning LLM Agents. arXiv:2512.20184, 2025. [10] T. Wyse, K. Bustos, Y. Volkova, M. Kleiman-Weiner. Commitment To Cooperation With Self-Negotiated Contracts. arXiv:2607.22750, 2026. [11] J. Geng, G. Neubig. Effective Strategies for Asynchronous Software Engineering Agents. arXiv:2603.21489, 2026. [12] H. Lyu, D. Zhang, M. Wu, X. Wei, H. Chen. CoAgent: Concurrency Control for Multi-Agent Systems. arXiv:2606.15376, 2026. [13] X. Li, Y. Liu, W. Chen, et al. SkillsBench. arXiv:2602.12670, 2026. [14] Z. Xiang, L. Zheng, Y. Li, et al. GuardAgent. *ICML*, 2025.

Positioning

When \ Where	Agent (instructions / skill)	Harness (enforced)
Before work	**this work (skill)**	Self-Negotiated Contracts [10]; **this work (gate)**
On change	—	CoAgent [12]
At integration	—	CAID [11]; **this work (end check)**
Any time	CooperBench messaging [1]	—

	Closest work [10]	This work
Why agents deviate	incentive (defect for gain)	honest drift (forget, mismatch)
What is checked	actions	content of outputs
Enforcement	always by environment	skill vs. harness, compared

[1] Khatua et al., COLM 2026. [10] Wyse et al., arXiv:2607.22750, 2026. [11] Geng & Neubig, arXiv:2603.21489, 2026. [12] Lyu et al., arXiv:2606.15376, 2026.

Model and Base Methods

Model

Agents	$a_1, \ldots, a_n$, different roles, own workspace only
Topic τ	shared choice; valid but incompatible options D_τ
Decision d	value from D_τ, committed in shared log L
Output o	what each agent produces after d
Conformance	$\mathrm{Conf}(o, d)$: does o follow d?
Failures	crash, context loss ($\approx$ crash-recovery), **silent divergence**
Assumption	L is reliable (single trusted store)

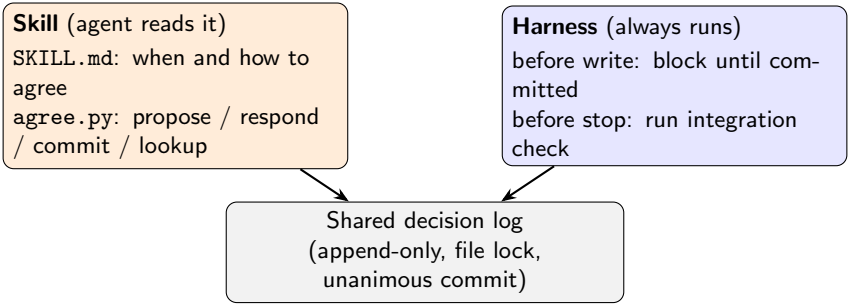
Base methods (existing)

What we take	Src
Validity, Agreement, Termination	[3]
Atomic commit: all accept $\Rightarrow$ commit, any veto $\Rightarrow$ abort	[4]
SMR: same decisions + determinism $\Rightarrow$ same outputs	[5]
Agent Skills: `SKILL.md` + scripts	[15]
Claude Code hooks: block before tool / stop	[16]
Two-role task, merged and tested	[1]

New: Conformance, since LLM agents are not deterministic

[1] Khatua et al., COLM 2026. [3] Lamport et al., TOPLAS 1982. [4] Bernstein et al., 1987. [5] Schneider, ACM CSUR 1990. [15] Agent Skills specification, https://agentskills.io/specification. [16] Claude Code docs: Hooks, https://code.claude.com/docs/en/hooks (both accessed Oct 2026).

Proposed Method: Overview and Interface



Operation	Meaning
`propose`(τ, d) / `respond`(r)	open a version; $r \in \{\text{accept}, \text{object}\}$, one object aborts
`commit`(τ) / `lookup`(τ)	all accepted $\Rightarrow$ append d / read latest d
`check`(o, τ) / `deviate`(τ)	run $\mathrm{Conf}(o, d)$ / announce instead of diverging silently

Proposed Method: Detail (Harness Variants and Conditions)

Harness variants

Variant	Behavior
Pre-write gate	cannot write own file until `interface` is committed
End check	cannot stop if the merged code fails the example
End + barrier	cannot stop until both are ready on the same file versions and the check passes

Conditions (10 runs each)

Condition	Where	When
None	—	—
Tool only (no procedure)	agent	any
Inline procedure (prompt)	agent	before
Skill (`SKILL.md`)	agent	before
Pre-write gate	harness	before
End check	harness	integration
End check + barrier	harness	integration
Skill + end check	both	both

Experiment Task and Environment

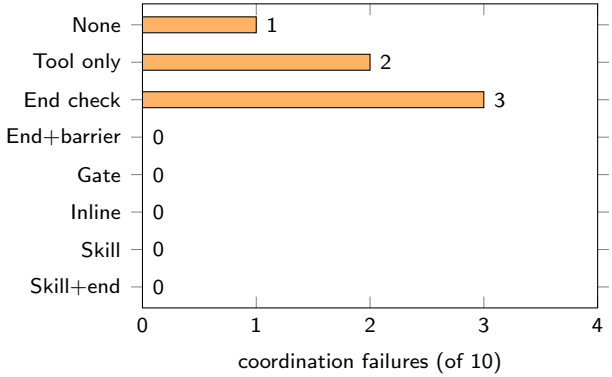
Task: Agent 1 writes `pricing.py`; Agent 2 writes `invoice.py` using it. Separate folders, message board only. Merged and graded by 7 hidden tests. Agents are not told what must be decided.

Hidden shared choice	Options
Price unit	cents / dollars / Decimal
Unknown item	`None` / `KeyError`
Discount	10 / 0.1 / "10%"

Environment

CPU	Intel Xeon @ 2.80 GHz, 2 vCPUs
GPU	none (models via Anthropic API)
Memory	7.8 GiB
OS	Ubuntu 24.04.5 LTS (cloud container)
Agent	Claude Code 2.1.312, headless (`claude -p`)
Model	`claude-haiku-5-5`, max 40 turns
Language	Python 3.13.16
Tools	Read, Write, Edit, Glob, Grep, limited Bash, Skill
Runs	8 conditions $\times$ 10, 5 trials in parallel

Results: Coordination Failures



Condition	Success
None	8/10
Tool only	8/10
Inline	10/10
Skill	8/10*
Gate	8/10†
End check	7/10
End + barrier	10/10
Skill + end	10/10

* both: own formatting bug

† 1 formatting bug, 1 permission issue

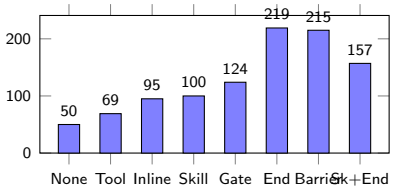
Procedure present 0/30 vs. absent 6/30 coordination failures (Fisher, one-sided $p = 0.012$; grouping chosen after the fact).

Results: What the Failures Looked Like

All 6 coordination failures: crossing proposals

Time	Agent	Action
0 s	Agent 2	proposes "unknown item ⇒ return `None`"
9 s	Agent 1	(before reading it) "done: unknown item ⇒ raise `KeyError`"
	Agent 2	adapts to `KeyError`, finishes
15 s	Agent 1	reads the old proposal, switches to `None`

Mean wall time (s)



- No single committed value ⇒ concurrent updates swap
- With a procedure, one value is committed first
- End check found the mismatch, but the partner had already exited

Discussion

What the pilot suggests

1. The **procedure** matters, not the tool (tool only = no change)
2. Skill format showed no extra effect over the same text in the prompt
3. Harness **timing** matters: an end check needs a barrier (like two-phase commit)
4. The cost is mostly time, not tokens

Why not Paxos or Raft? One trusted log, no message loss, and with 2 agents a majority is everyone; failures came from no commit point, not crashes.

Limitations

- 10 runs per condition, one model, one synthetic task; baseline already 8/10
- Two experimenter bugs found and fixed (gate blocked the agreement tool; permission patterns)

Question List and Next Week

Questions

1. **Novelty**: normative MAS already monitors compliance. Is "agents negotiate d themselves + output conformance" enough?
2. **Commit rule**: unanimous, owner decides, or majority? Default or variable?
3. **Skill vs. inline**: if the format adds nothing, should the axis be "procedure vs. enforcement"?
4. **Task realism**: is a synthetic task acceptable, or must I use an existing benchmark (e.g., CooperBench)?
5. **Log model**: is a single trusted log acceptable, or should the log be replicated?

Next week

- Confirmatory run (pre-registered): none / inline / skill, 30 runs each
- Harder task: longer work, mid-task spec change, 3 agents
- Second use case with the same skill; fix permission patterns

References

[1] A. Khatua, H. Zhu, P. Tran, et al. CooperBench: Why Coding Agents Cannot be Your Teammates Yet. *COLM*, 2026.

[2] Z. Wang, Y. Zhang, D. Lee, N. Baracaldo, Y. Yang. Lost in Context Compaction: When LLM Systems Drop Side Constraints. arXiv:2608.11242, 2026.

[3] L. Lamport, R. Shostak, M. Pease. The Byzantine Generals Problem. *ACM TOPLAS* 4(3):382–401, 1982.

[4] P. A. Bernstein, V. Hadzilacos, N. Goodman. *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, 1987.

[5] F. B. Schneider. Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial. *ACM Computing Surveys* 22(4):299–319, 1990.

[6] M. P. Singh. An Ontology for Commitments in Multiagent Systems: Toward a Unification of Normative Concepts. *Artificial Intelligence and Law* 7:97–113, 1999.

[7] D. Grossi, H. Aldewereld, F. Dignum. Ubi Lex, Ibi Poena: Designing Norm Enforcement in E-Institutions. *COIN*, LNCS, 2007.

[8] F. Berdoz, L. Rugli, R. Wattenhofer. Can AI Agents Agree? arXiv:2603.01213, 2026.

[9] C. Ruan, Y. Wang, Z. Shi, J. Li. Reaching Agreement Among Reasoning LLM Agents. arXiv:2512.20184, 2025.

[10] T. Wyse, K. Bustos, Y. Volkova, M. Kleiman-Weiner. Commitment To Cooperation With Self-Negotiated Contracts. arXiv:2607.22750, 2026.

[11] J. Geng, G. Neubig. Effective Strategies for Asynchronous Software Engineering Agents. arXiv:2603.21489, 2026.

[12] H. Lyu, D. Zhang, M. Wu, X. Wei, H. Chen. CoAgent: Concurrency Control for Multi-Agent Systems. arXiv:2606.15376, 2026.

[13] X. Li, Y. Liu, W. Chen, et al. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. arXiv:2602.12670, 2026.

[14] Z. Xiang, L. Zheng, Y. Li, et al. GuardAgent: Safeguard LLM Agents by a Guard Agent via Knowledge-Enabled Reasoning. *ICML*, 2025.

[15] Agent Skills specification. https://agentskills.io/specification (accessed Oct 2026).

[16] Claude Code documentation: Hooks. https://code.claude.com/docs/en/hooks (accessed Oct 2026).